

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – High (Scenario Based Assessment)

Course Code:	CSA0901	Course Title:	Programming in Java
CO Assessed:	CO2 – Precision (BL3,BL4)	Assessment:	ASSESSMENT TOOL 1 - High (Scenario Based Assessment)
Weightage:		Total Marks:	100
CO1 Statement:	Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations (BL3,BL4)		
Student Name:	ANUSREE A	Reg. No.:	1911260033
Section / Batch:	SLOT B	Date:	27/07/2026

Code of Conduct

I ANUSREE A / 1911260033 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- This test contains 10 questions, each carrying 10 marks. Total: 100 Marks.
- No negative marking. Un answered questions carry zero marks.
- No calculators or electronic devices permitted.
- Duration: 90 minutes.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Application of Concepts	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Analytical & Decision-Making Skills	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Solution Design & Approach	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Clarity, Justification & Presentation	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

ANNEXURE

Scenario Based Assessment

Q1: Student Registration System

Suppose that a university system stores student data, where each student must be unique and can enroll in multiple courses.

- (a) Enumerate three Collection interfaces suitable for this system.
- (b) Design a Collection structure (diagram/representation) to store students and their enrolled courses.
- (c) Write a Java program using HashSet and HashMap with Generics to:
 - Add students
 - Map students to courses
 - Display data using Iterator

PROGRAM:

```
import java.util.*;

class Student {
    int id;
    String name;

    Student(int id, String name) {
        this.id = id;
        this.name = name;
    }
}
```

```

    }

    // Ensures uniqueness based on student ID
    @Override
    public int hashCode() {
        return Objects.hash(id);
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj)
            return true;
        if (!(obj instanceof Student))
            return false;
        Student s = (Student) obj;
        return this.id == s.id;
    }

    @Override
    public String toString() {
        return id + " - " + name;
    }
}

public class StudentRegistrationSystem {
    public static void main(String[] args) {

        // HashSet to store unique students
        HashSet<Student> students = new HashSet<>();

        Student s1 = new Student(101, "Alice");
        Student s2 = new Student(102, "Bob");
        Student s3 = new Student(103, "Charlie");

        // Add students
        students.add(s1);
        students.add(s2);
        students.add(s3);

        // HashMap to map students to courses
        HashMap<Student, List<String>> courseMap = new HashMap<>();
    }
}

```

```

        courseMap.put(s1, Arrays.asList("Java", "Database",
"Mathematics"));
        courseMap.put(s2, Arrays.asList("Python", "Networking"));
        courseMap.put(s3, Arrays.asList("Java", "Operating Systems"));

// Display students using Iterator
System.out.println("Student Registration Details:");
Iterator<Student> iterator = students.iterator();

while (iterator.hasNext()) {
    Student student = iterator.next();
    System.out.println("Student: " + student);
    System.out.println("Courses: " + courseMap.get(student));
    System.out.println();
}
}
}

```

OUTPUT:

```

Student Registration Details:
Student: 101 - Alice
Courses: [Java, Database, Mathematics]

Student: 102 - Bob
Courses: [Python, Networking]

Student: 103 - Charlie
Courses: [Java, Operating Systems]

```

Q2: E-Commerce Cart System

Suppose that an online shopping system maintains a cart where products are added, removed, and displayed in order.

- (a) List three Collection classes applicable for cart management.
- (b) Represent how cart items are stored using a List-based structure.
- (c) Write a Java program using ArrayList and Iterator to:
 - Add items
 - Remove an item
 - Display all items

PROGRAM:

```

import java.util.ArrayList;

```

```

import java.util.Iterator;

public class EcommerceCart {
    public static void main(String[] args) {

        // Create an ArrayList
        ArrayList<String> cart = new ArrayList<>();

        // Add items
        cart.add("Laptop");
        cart.add("Mouse");
        cart.add("Keyboard");
        cart.add("Headphones");

        // Remove an item
        cart.remove("Mouse");

        // Display all items using Iterator
        System.out.println("Items in Shopping Cart:");

        Iterator<String> iterator = cart.iterator();

        while (iterator.hasNext()) {
            System.out.println(iterator.next());
        }
    }
}

```

OUTPUT:

```

Items in Shopping Cart:
Laptop
Keyboard
Headphones

```

Q3: Library Management System

Suppose that a library stores books with unique ISBN numbers and needs fast retrieval.

- (a) Name three Map implementations in Java.
- (b) Design a schema-like representation using Map for storing ISBN and book details.
- (c) Write a Java program using HashMap to:

- Insert book details
- Retrieve a book
- Display all books

PROGRAM:

```
import java.util.HashMap;
import java.util.Map;

public class LibraryManagementSystem {

    public static void main(String[] args) {

        // Create a HashMap to store ISBN and Book Details
        HashMap<String, String> library = new HashMap<>();

        // Insert book details
        library.put("9780134685991", "Effective Java - Joshua Bloch");
        library.put("9780132350884", "Clean Code - Robert C. Martin");
        library.put("9780596009205", "Head First Java - Kathy Sierra");

        // Retrieve a book using ISBN
        String isbn = "9780132350884";

        System.out.println("Retrieved Book:");
        System.out.println("ISBN : " + isbn);
        System.out.println("Book : " + library.get(isbn));

        // Display all books
        System.out.println("\nLibrary Books:");
        for (Map.Entry<String, String> entry : library.entrySet()) {
            System.out.println("ISBN : " + entry.getKey());
            System.out.println("Book : " + entry.getValue());
            System.out.println();
        }
    }
}
```

OUTPUT:

```
Retrieved Book:
ISBN : 9780132350884
Book : Clean Code - Robert C. Martin
```

Library Books:

ISBN : 9780596009205

Book : Head First Java - Kathy Sierra

ISBN : 9780132350884

Book : Clean Code - Robert C. Martin

ISBN : 9780134685991

Book : Effective Java - Joshua Bloch

Q4: Employee Data Processing (10 Marks)

Suppose that a company processes employee records and filters employees based on salary.

(a) List three advantages of Generics in Collections.

(b) Represent how employee data is stored using List with Generics.

(c) Write a Java program using Iterator to:

- Store employee objects
- Display employees with salary > 50,000

PROGRAM:

```
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

class Employee {
    int id;
    String name;
    double salary;

    Employee(int id, String name, double salary) {
        this.id = id;
        this.name = name;
        this.salary = salary;
    }
}

public class EmployeeDataProcessing {

    public static void main(String[] args) {

        // Create a List of Employee objects
```



```

List<Employee> employees = new ArrayList<>();

// Store employee objects
employees.add(new Employee(101, "Alice", 45000));
employees.add(new Employee(102, "Bob", 60000));
employees.add(new Employee(103, "Charlie", 75000));
employees.add(new Employee(104, "David", 50000));

// Display employees with salary > 50000
System.out.println("Employees with Salary > 50000:");

Iterator<Employee> iterator = employees.iterator();

while (iterator.hasNext()) {
    Employee emp = iterator.next();

    if (emp.salary > 50000) {
        System.out.println("ID: " + emp.id);
        System.out.println("Name: " + emp.name);
        System.out.println("Salary: " + emp.salary);
        System.out.println();
    }
}
}
}

```

OUTPUT:

Employees with Salary > 50000:

ID: 102
 Name: Bob
 Salary: 60000.0

ID: 103
 Name: Charlie
 Salary: 75000.0

Q5: Online Voting System (10 Marks)

Suppose that an online voting system ensures one vote per user and counts votes efficiently.

- (a) Identify three suitable Collection types for this system.
- (b) Design a structure using Set and Map for voters and vote counting.
- (c) Write a Java program using HashSet and HashMap to:
- Record votes
 - Prevent duplicate voting
 - Display results

PROGRAM:

```
import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;

public class OnlineVotingSystem {

    public static void main(String[] args) {

        // HashSet to store unique voters
        HashSet<String> voters = new HashSet<>();

        // HashMap to store vote count
        HashMap<String, Integer> voteCount = new HashMap<>();

        // Record votes
        castVote(voters, voteCount, "V001", "Alice");
        castVote(voters, voteCount, "V002", "Bob");
        castVote(voters, voteCount, "V003", "Alice");
        castVote(voters, voteCount, "V001", "Charlie"); // Duplicate
vote
        castVote(voters, voteCount, "V004", "Charlie");

        // Display results
        System.out.println("\nVoting Results:");
        for (Map.Entry<String, Integer> entry : voteCount.entrySet()) {
            System.out.println(entry.getKey() + " : " +
entry.getValue() + " vote(s)");
        }
    }

    // Method to record votes and prevent duplicate voting
    public static void castVote(HashSet<String> voters,
                                HashMap<String, Integer> voteCount,
                                String voterId,
```

```
        String candidate) {

    if (voters.contains(voterId)) {
        System.out.println("Voter " + voterId + " has already
voted.");
    } else {
        voters.add(voterId);
        voteCount.put(candidate, voteCount.getDefault(candidate,
0) + 1);
        System.out.println("Vote recorded for " + candidate + " by
" + voterId);
    }
}
}
```

OUTPUT:

```
Vote recorded for Alice by V001
Vote recorded for Bob by V002
Vote recorded for Alice by V003
Voter V001 has already voted.
Vote recorded for Charlie by V004
```

```
Voting Results:
Bob : 1 vote(s)
Alice : 2 vote(s)
Charlie : 1 vote(s)
```